

Compiladores, 2025/2026

Aula prática 2

Fernando Lobo

Exercício 1

(Retirado do livro do dragão, *Compilers: Principles, Techniques, and Tools*. Edição de 1986, pág 267, Ex. 4.1)

Considere a seguinte gramática:

$$\begin{aligned} S &\rightarrow (L) \mid a \\ L &\rightarrow L, S \mid S \end{aligned}$$

- a) Indique os símbolos terminais, não terminais, e o símbolo de partida.
- b) Especifique (usando papel e lápis) árvores sintáticas para as seguintes frases:
 - i) (a, a)
 - ii) $(a, (a, a))$
 - iii) $(a, ((a, a), (a, a)))$
- c) Construa uma derivação mais à esquerda para cada frase da alínea b)
- d) Construa uma derivação mais à direita para cada frase da alínea b)
- e) Especifique a gramática num ficheiro (.g4) usando o ANTLR e verifique que as árvores sintáticas obtidas pelo ANTLR para os inputs especificados na alínea b) coincidem com as árvores sintáticas que obteve em papel. (Note que o ANTLR usa uma convenção que é o contrário daquela que é utilizada no livro do Dragão e nos slides de apoio às aulas teóricas. No livro e nos slides, os símbolos não terminais são escritos em maiúsculas e os símbolos terminais (tokens) em minúsculas. Em ANTLR é ao contrário: símbolos não terminais são em minúsculas, símbolos terminais são em maiúsculas.)

Exercício 2

[Retirado do livro do dragão, *Compilers: Principles, Techniques, and Tools*, Edição de 1986, pág 267, Ex. 4.2))

Considere a seguinte gramática:

$$S \rightarrow aSbS \mid bSaS \mid \epsilon$$

- a) Mostre que a gramática é ambígua, especificando duas derivações mais à esquerda distintas que geram a frase **abab**
- b) Construa as derivações mais à direita correspondentes.
- c) Construa as árvores sintáticas correspondentes para **abab**.
- d) Descreva em português corrente qual a linguagem gerada por esta gramática

Exercício 3

Considere a gramática Expr.g4 disponível na tutoria.

- a) Teste a gramática com os seguintes inputs, e observe a estrutura das árvores sintáticas resultantes. O que pode concluir?
 - `a + b + c + d`
 - `a * b + c + d`
 - `a + b * c - d`
 - `a + b * c / d`
 - `(a + b) * (c + d)`
 - `total + base * altura33`
- b) Substitua a primeira alternativa do símbolo **expr** que contém **PLUS|MINUS|TIMES|DIV** por 4 alternativas separadas, uma para cada operador, e volte a testar os inputs. O que pode concluir?
- c) Coloque agora 2 alternativas em vez das 4. Uma com os operadores **PLUS|MINUS** em primeiro lugar e outra com os operadores **TIMES|DIV** em segundo lugar, e volte a testar os inputs anteriores.
- d) Troque a ordem das duas alternativas anteriores e volte a testar os inputs.

- e) Baseado no que observou na resolução das alíneas anteriores, o que pode concluir relativamente ao modo como o ANTLR resolve a ambiguidade da gramática?
- f) Acrescente à gramática o operador de potência ($\wedge$). Este operador deve ter maior precedência que a multiplicação e divisão, que por sua vez têm maior precedência que a soma e subtração. Por exemplo: $1+2^4$ deve ser interpretado como $1+(2^4) = 1+16 = 17$. Note ainda que o operador de potência deve ter associatividade à direita. Por exemplo, 2^3^2 deve ser interpretado como $2^(3^2)$ e não como $(2^3)^2$.

Teste a gramática resultante com inputs apropriados e certifique-se que as árvores sintáticas correspondentes refletem a prioridade e associatividade esperada para os operadores.

Por defeito o ANTLR usa a associatividade à esquerda. Se quisermos usar a associatividade à direita, podemos especificar em ANTLR usando `<assoc=right>`. Por exemplo, com a regra:

`expr : expr PLUS expr`

o input $2+3+4+5$ seria interpretado como $((2+3)+4)+5$. Porém, se a regra for

`expr : <assoc=right> expr PLUS expr`

o input $2+3+4+5$ já seria interpretado como $2+(3+(4+5))$.